70-467 Machine Learning for Business Analytics

Session 06: Tree-based methods

Professor: Andrés Castaño Zuluaga

Carnegie Mellon University

BUSINESS
CMUQ

How many bikes will Seoul riders rent in the next hour?

Ttareungyi (Seoul Bike) is the city of Seoul's public bike-share system. A rider rents a bike at a rental station and returns it to any station. A rider who finds no bike has to wait or travel another way.

High or Low?

A class. An hour is **High** when it has at least 500 rentals.

How many rentals?

A number: the rentals in that hour.

The data: 8,465 open hours from 1 December 2017 to 30 November 2018. 52.0% of them are High.

Can one method answer both questions?

What do we know about each hour?

Variable	Type	Unit	Description
Rented Bike Count	integer	bikes	Bikes rented in the hour. The outcome. High when at least 500.
Hour	integer	0 to 23	Hour of the day
Temperature	numeric	°C	Temperature in the hour
Humidity	integer	%	Humidity in the hour
Wind speed	numeric	m/s	Wind speed in the hour
Visibility	integer	10 m	Visibility in the hour
Solar Radiation	numeric	MJ/m^2	Solar radiation in the hour
Rainfall	numeric	mm	Rainfall in the hour
Snowfall	numeric	cm	Snowfall in the hour
Seasons	text	none	Winter, Spring, Summer, or Autumn
Holiday	text	none	Holiday or No Holiday

One observation is one open hour. The file also has the date, the dew point and Functioning Day. We do not use them.

> Which of these would the city know before the hour starts?

Data: Seoul Bike Sharing Demand, University of California, Irvine (UCI) Machine Learning Repository, doi:10.24432/C5F62R, licence CC BY 4.0.

Where are we on the course roadmap?

	Description	Tools	Use
PREDICT & EVALUATE			
Linear regression	Predict a number	OLS	
Metrics	Size of the errors	RMSE, R^2	
Train/test + leakage	Predict without fooling ourselves	Holdout	
Regularization	Shrink complexity	Ridge, lasso	
Cross-validation	Choose λ without touching test	k-fold	
CLASSIFY & DECIDE			
Logistic	A probability, not a class	Log-odds	
Thresholds	A score is not an action	ROC, threshold	
FLEXIBLE & UNSUPERVISED			
Trees	**When linearity fails**	**Splits**	**Split the hours into simple regions.**
Forests / boosting	One ensemble	Random forest, boosting	
PCA	Compress X, no Y	PCs	
Clustering	Segment without Y	K-means	

> What will we do in this cell?

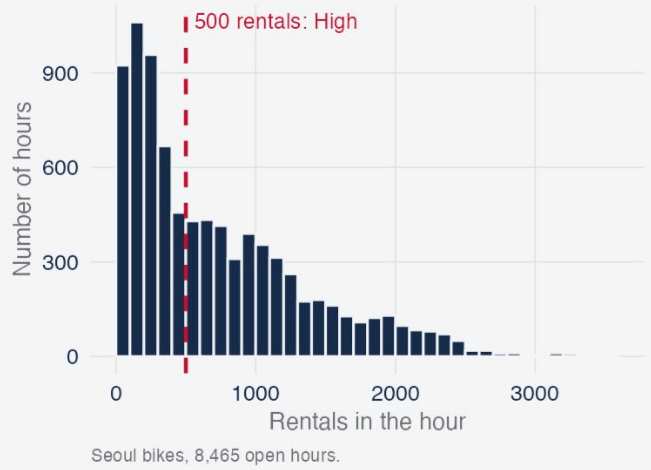
What we will do

1. What is a decision tree classifier?
2. How are decision trees computed from the training data?
3. What are the big advantages and disadvantages of decision trees?
4. How can we minimize overfitting in tree models?
5. How can many trees predict better than one? Bagging, random forests and boosting.
6. Which model holds up on hours it has not seen?

What is in the Seoul bike data?

```r
raw <- read.csv("data/SeoulBikeData.csv", fileEncoding = "latin1", check.names = FALSE)  # degree signs in the headers
names(raw) <- gsub("^_|_$", "", gsub("[^A-Za-z0-9]+", "_", names(raw)))                  # Temperature(C) becomes Temperature_C
bikes <- subset(raw, Functioning_Day == "Yes")                                           # keep the hours the system is open
bikes$High <- factor(ifelse(bikes$Rented_Bike_Count >= 500, "High", "Low"), levels = c("Low", "High"))  # busy hour: 500+ rentals
table(bikes$High)                                                                        # quiet and busy hours
```

```
 Low High
4066 4399
```



Seoul bikes, 8,465 open hours.

- **One observation is one hour.** 8,760 hours, 1 December 2017 to 30 November 2018.
- **Closed hours are dropped.** The system was closed for 295 hours, all with 0 rentals. That leaves 8,465 open hours on 353 days.
- **Rentals per hour:** median 542, mean 729, maximum 3,556. 4,399 hours are High and 4,066 are Low.
- **10 predictors:** hour, temperature, humidity, wind speed, visibility, solar radiation, rainfall, snowfall, season and holiday.

Why is the mean above the median, and what would predicting 729 for every hour miss?

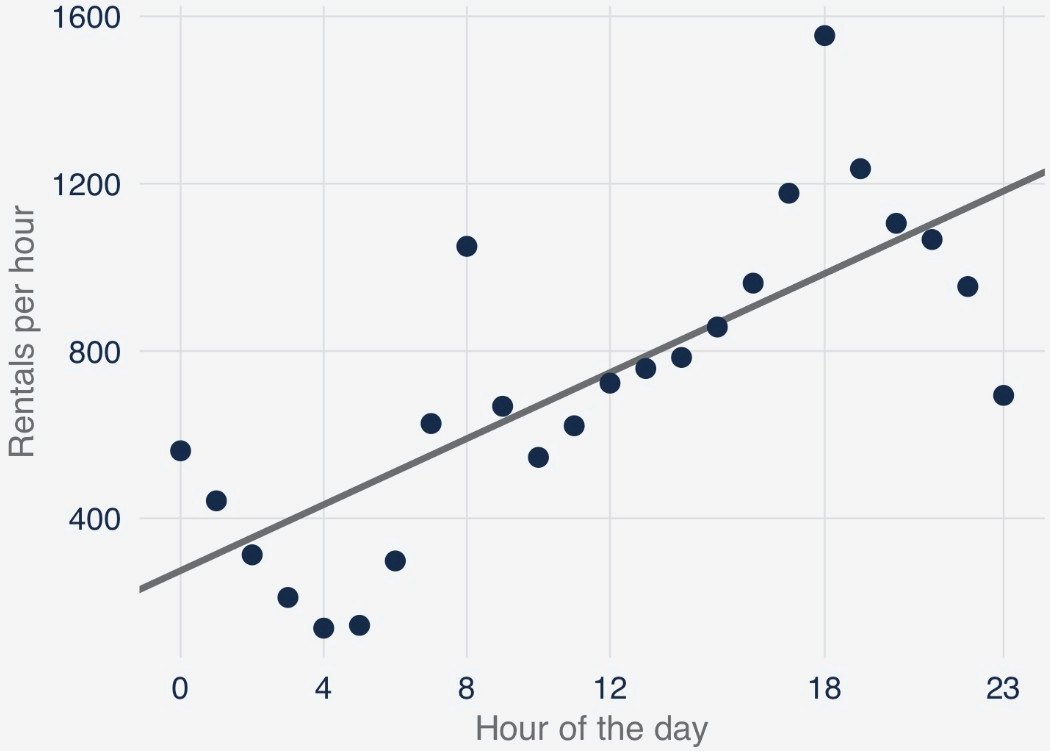
What would the methods we know predict for 8 am tomorrow?

```r
hour8 <- data.frame(Hour = 8, Temperature_C = 25, Rainfall_mm = 0, Holiday = "No Holiday")  # 8 am tomorrow
lin <- lm(count ~ Hour + Temperature_C + Rainfall_mm + Holiday, data = bikes)                # rentals: a number
logit <- glm(High ~ Hour + Temperature_C + Rainfall_mm + Holiday, data = bikes, family = binomial)  # P(High)
print(data.frame(rentals = round(predict(lin, hour8)),                                       # both predictions
                 P_High = round(predict(logit, hour8, type = "response"), 2)), row.names = FALSE)
```

```
rentals P_High
    971   0.86
```

- **The hour on the sheet:** 8 am, 25°C, no rain, not a holiday. Both models are fit on all 8,465 open hours.
- **Linear regression predicts 971 rentals. Logistic regression gives P(High) = 0.86.**
- **The 29 open hours like it** (8 am, 23 to 27°C, no rain, not a holiday) **averaged 1,590 rentals, and all 29 were High.**

How do rentals change over the day?



Dots: mean rentals by hour, all open hours. Line: lm on hour.

- **Mean rentals by hour:** 137 at 4 am, 1,050 at 8 am and 1,554 at 6 pm.
- **The line has one slope:** 39.5 rentals for each hour later in the day. It predicts 433 at 4 am, 591 at 8 am and 985 at 6 pm.
- **It misses the night low and both peaks.**
- **Your rules on the sheet do something else.** "If it is 8 am, expect about ... rentals" puts the hours into groups and predicts one number for each group.

If the prediction is a set of rules, how do we choose the first rule?

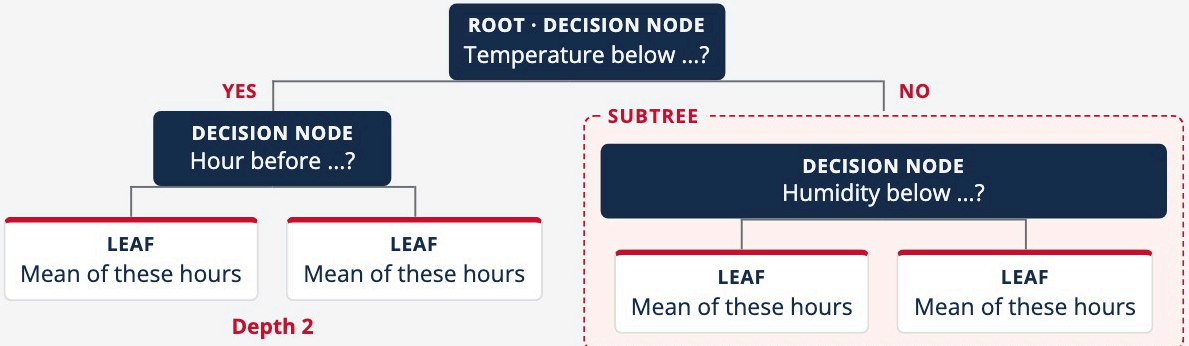
How do we keep test hours out of the estimation?

```
set.seed(467)                                          # same split every run
split <- initial_split(bikes, prop = 0.70, strata = High)      # 70/30, same share of High in each part
train <- training(split)                               # estimate and choose every setting here
test  <- testing(split)                                # only score here
c(nrow(train), nrow(test))                             # hours in each part
```

```
[1] 5925 2540
```

- **One split for everything.** 70% of the hours train and 30% test, with seed 467. Both outcomes and every model in both sittings use the same 5,925 training and 2,540 test hours.
- **Stratified on High.** 52.0% of the hours are High in both parts.
- **Training hours choose every setting.** The size of a tree is chosen on the 5,925 training hours.
- **Test hours only score.** Each model's test score is shown when the model appears, and all the scores are gathered in one comparison at the end.

What are the parts of a tree?

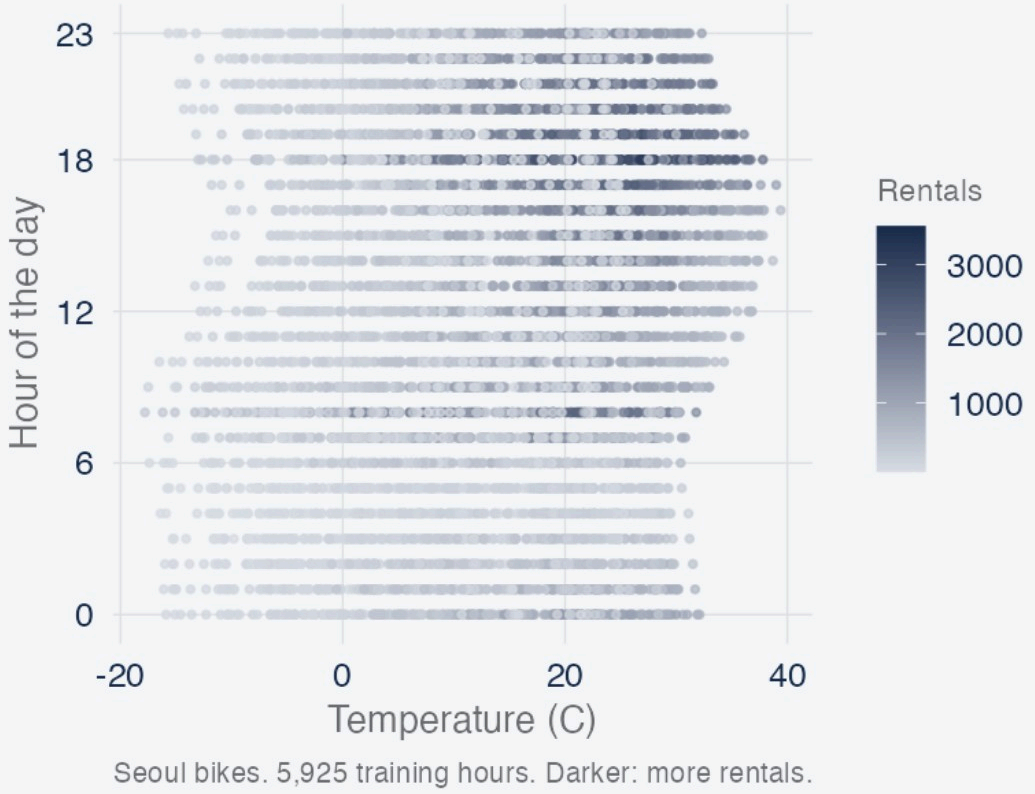


Example questions. Not the tree the data chooses.

- **The root is a decision node.** All 5,925 training hours start there. It asks one yes/no question about one predictor.
- **Every hour takes one path** and ends at one leaf.
- **A leaf predicts one number.** For rentals, the mean of the training hours that reach it. For High or Low, the share of High hours.
- **Depth** is the number of questions on that path. **A subtree** is everything below one node.

A new hour reaches a leaf. What does the city get from that leaf?

Where does your first rule draw the line?



Seoul bikes. 5,925 training hours. Darker: more rentals.

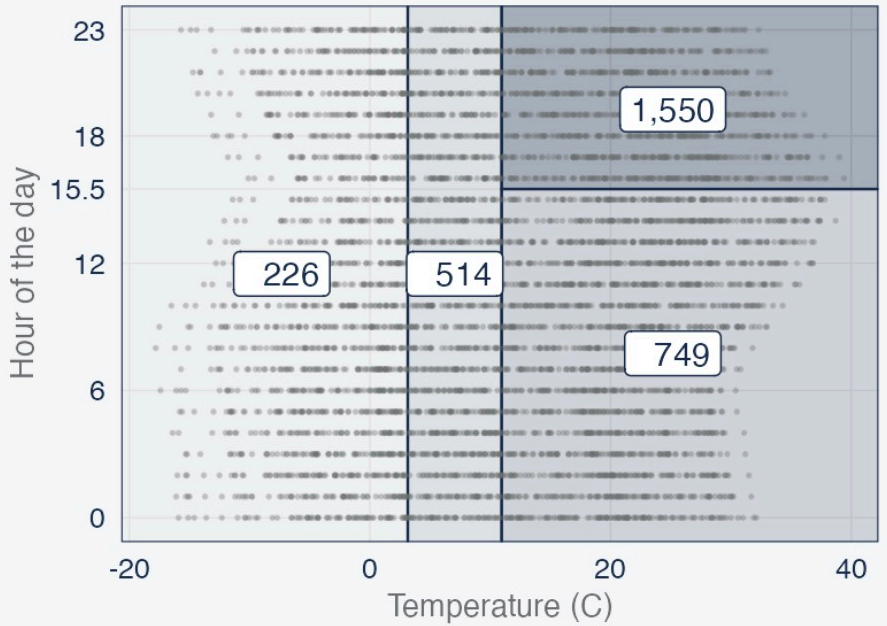
- **Each dot is one training hour**, placed by temperature and hour of the day. Darker dots had more rentals.
- **Your first rule is one straight line, vertical or horizontal:** "if the temperature is below …, expect about … rentals." Where would you put it so the two sides differ most in rentals?
- **Baseline before any line:** predict the training mean, 732.5 rentals, for every hour.
- **RMSE (root mean squared error)** is the typical size of a prediction error, in rentals. The baseline's test RMSE is 630.8.

With only temperature and the hour, where would you draw the first line?

What does a regression tree do with the hours?

```
reg2 <- rpart(count ~ Hour + Temperature_C, data = train,      # two predictors, so we can draw it
              method = "anova",                                # regression tree: leaf = mean rentals
              control = rpart.control(maxdepth = 2, cp = 0, minsplit = 20, minbucket = 7, xval = 0))  # two levels
rmse_vec(test$count, predict(reg2, test))                      # test RMSE, rentals
```

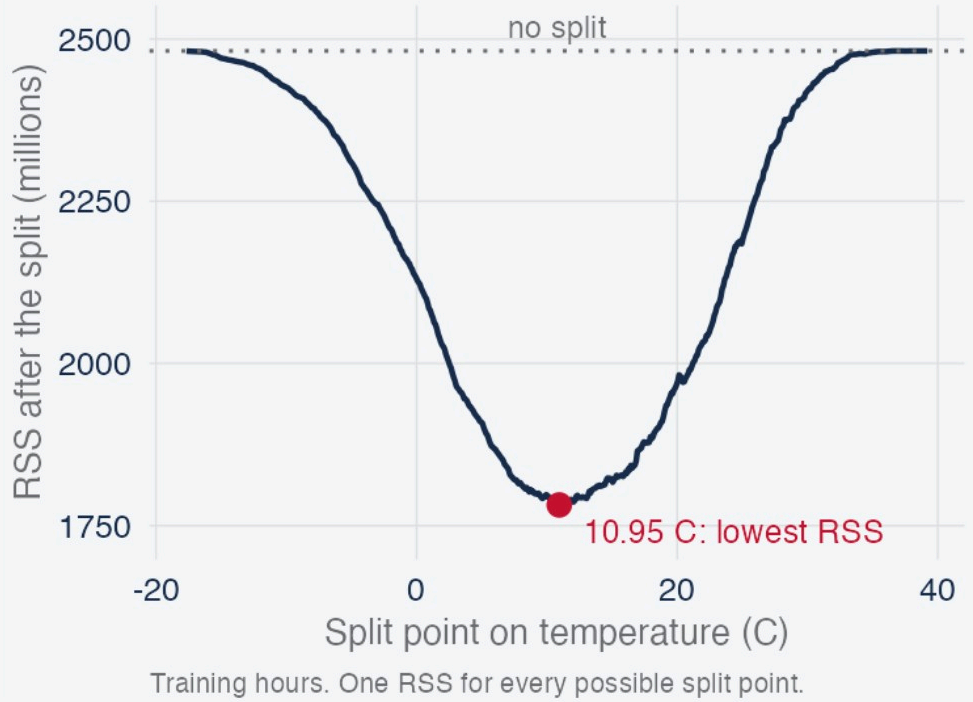
```
[1] 467.3371
```



Labels: mean rentals in each region (training hours).

- **First split: temperature at 10.95°C.**
- **One more split on each side.** The colder side splits at 3.15°C. The warmer side splits on the hour: `Hour < 15.5` means through 3pm against from 4pm.
- **Each rectangle predicts its mean:** 226, 514, 749 and 1,550 rentals per hour.
- **Test RMSE 467.3**, against 630.8 for the mean.

How does the tree choose where to split?



Training hours. One RSS for every possible split point.

- **Try every split point and keep the one with the lowest RSS** (residual sum of squares). The prediction on each side is the mean of the training hours on that side:

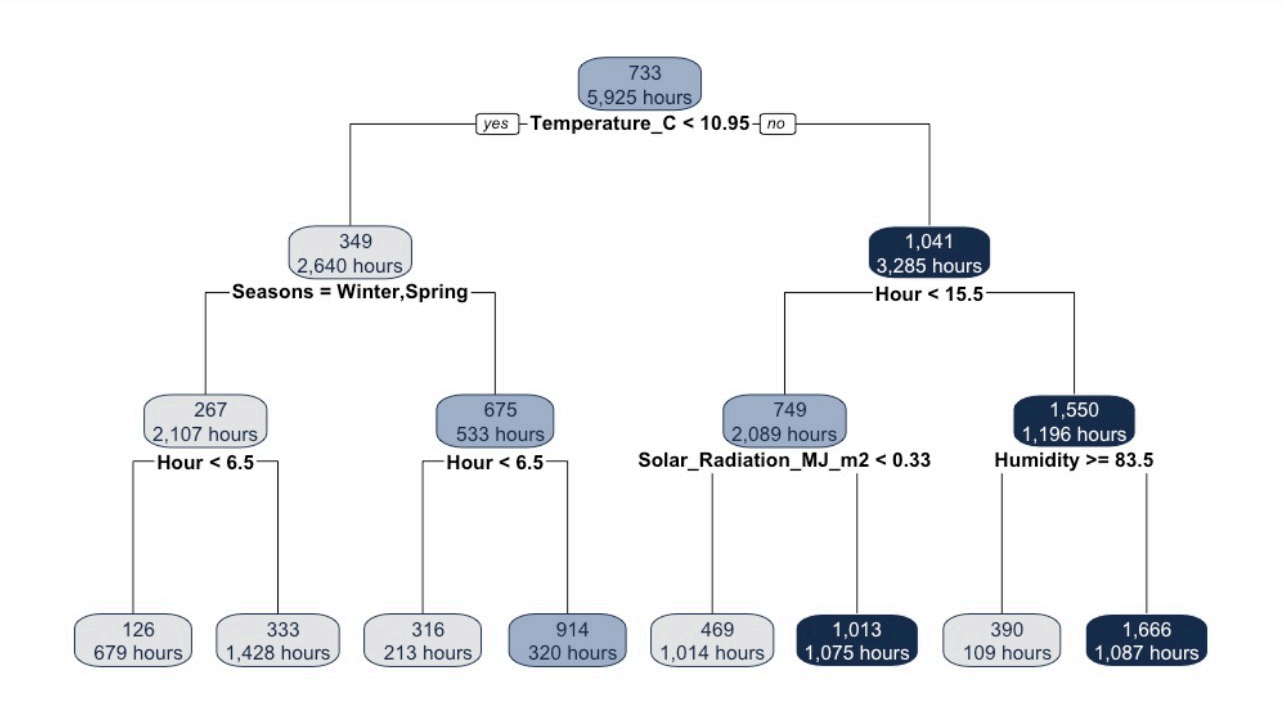
$$\text{RSS} = \sum_{i \in R_1} (y_i - \hat{y}_{R_1})^2 + \sum_{i \in R_2} (y_i - \hat{y}_{R_2})^2$$

- **Read it:** with no split, RSS is 2,481.6 million. At 10.95°C, 2,640 colder hours average 349.1 rentals and 3,285 warmer hours average 1,040.6. RSS falls to 1,781.9 million, so 28.2% of it is removed.
- **Greedy:** take the best split now, with no look-ahead, then repeat inside each side.

How do we read a regression tree?

```
reg3 <- rpart(count ~ ., data = train[, c("count", predictors)],   # all 10 predictors
              method = "anova",                                    # regression tree
              control = rpart.control(maxdepth = 3, cp = 0, minsplit = 20, minbucket = 7, xval = 0))  # three levels, no penalty
rmse_vec(test$count, predict(reg3, test))                          # test RMSE, rentals
```

```
[1] 385.6382
```



- **7 splits, 8 leaves**, using temperature, season, hour, solar radiation and humidity.
- **Quietest leaf:** below 10.95°C, winter or spring, through 6am. 126 rentals per hour (679 hours).
- **Busiest leaf:** 10.95°C or warmer, from 4pm, humidity 83% or less. 1,666 rentals per hour (1,087 hours).
- **Test RMSE 385.6**, against 467.3 with two predictors.

What changes when the outcome is High or Low?

- **The tree still asks one yes/no question at a time** and sends each hour down one path.
- **A leaf no longer predicts a mean number of rentals.** It predicts the more common class. Its P(High) is the share of High training hours in the leaf.
- **The split can no longer be chosen by the residual sum of squares.** A class has no residual in rentals.

> The residual sum of squares is gone. How do we score a split?

Why not score a split by the error rate?

The leaf predicts the most common class. The error rate is one minus that share.

$$E = 1 - \max_k(\hat{p}_{mk})$$

- $\hat{p}_{mk}$ **is the share of training hours in class** k**.** At the root, High is 0.520, so the leaf predicts High.
- $E = 1 - 0.520 = 0.480$**.** The other 48.0% are Low. The leaf called them High. If every hour is High, $E = 0$.
- **100 hours, 80 High.** $E = 0.20$. Two groups of 50: 35 High and 45 High. Both predict High. Wrong hours: 15 and 5. Still 20 wrong. E is still 0.20.

35 High and 45 High are different. E is still 0.20. Why won't the tree split here?

What does Gini measure?

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$$

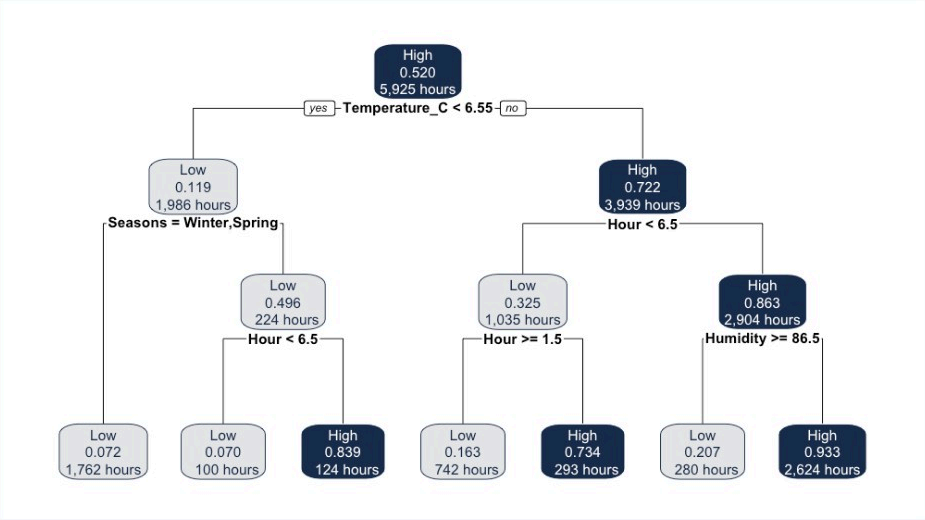
- A measure of total variance across the K classes.
- A measure of node purity.
- $\hat{p}_{mk}$: proportion of training observations in the mth region that belong to the kth class.
- G is low if the $\hat{p}_{mk}$ are close to 0 or 1, meaning node purity.

What does the depth-3 tree say in plain words?

```r
auc_vec <- function(truth, p) roc_auc_vec(truth, p, event_level = "second")   # AUC from yardstick; High is the event
acc_vec <- function(truth, p) mean((p >= 0.5) == (truth == "High"))          # accuracy: label High when P(High) >= 0.5
```

```r
cls3 <- rpart(High ~ ., data = train[, c("High", predictors)],     # classify High vs Low
              method = "class", parms = list(split = "gini"),      # Gini index; a leaf predicts its most common class
              control = rpart.control(maxdepth = 3, cp = 0, minsplit = 20, minbucket = 7, xval = 0))  # three levels
p_cls3 <- predict(cls3, test, type = "prob")[, "High"]             # P(High) = share of High hours in the leaf
round(c(accuracy = acc_vec(test$High, p_cls3), AUC = auc_vec(test$High, p_cls3)), 3)  # test scores
```

```
accuracy      AUC
   0.907    0.927
```



- **Root:** Temperature below 6.55°C?
- **Left, cold:** Winter and Spring are Low (0.072, 1,762 hours). Autumn is Low through 6am (0.070) and High from 7am (0.839).
- **Right, through 6am:** High at midnight and 1am (0.734); Low from 2am to 6am (0.163).
- **Right, from 7am:** High (0.933, 2,624 hours) unless humidity is 87% or more (0.207).
- **Test hours:** accuracy 0.907, AUC 0.927.

What happens to the error rate as the tree grows?



Seoul bikes, classification of High. 0 splits: predict High for every hour.

- **Vertical axis:** share of hours classified wrong. With no splits every hour is High, so the error is 0.480.
- **Grey is training; red is test.** Grown with Gini; scored with the error rate.
- **Grey keeps falling,** from 0.480 to 0.052. Later splits chase noise in the training hours.
- **Red falls, then rises:** 0.0929 at 6 splits (the depth-3 tree), 0.0768 at 29, and 0.0854 at 95.

How do we choose a smaller tree without looking at the test hours?

How does cost-complexity pruning shrink the tree?

```r
set.seed(467)                                                  # the 10 CV folds are random
full <- rpart(High ~ ., data = train[, c("High", predictors)],      # grow the full tree
              method = "class",
              control = rpart.control(maxdepth = 30, cp = 0, minsplit = 20, minbucket = 7, xval = 10))  # no penalty, 10-fold CV
round(full$cptable[c(1:4, 14, 16), ], 5)                       # some subtrees from the table
```

```
        CP nsplit rel error  xerror    xstd
1  0.53127      0   1.00000 1.00000 0.01351
2  0.12755      1   0.46873 0.47294 0.01133
3  0.05762      2   0.34118 0.35629 0.01019
4  0.04814      3   0.28356 0.30569 0.00957
14 0.00123     21   0.15847 0.18517 0.00770
16 0.00094     29   0.14933 0.18201 0.00764
```

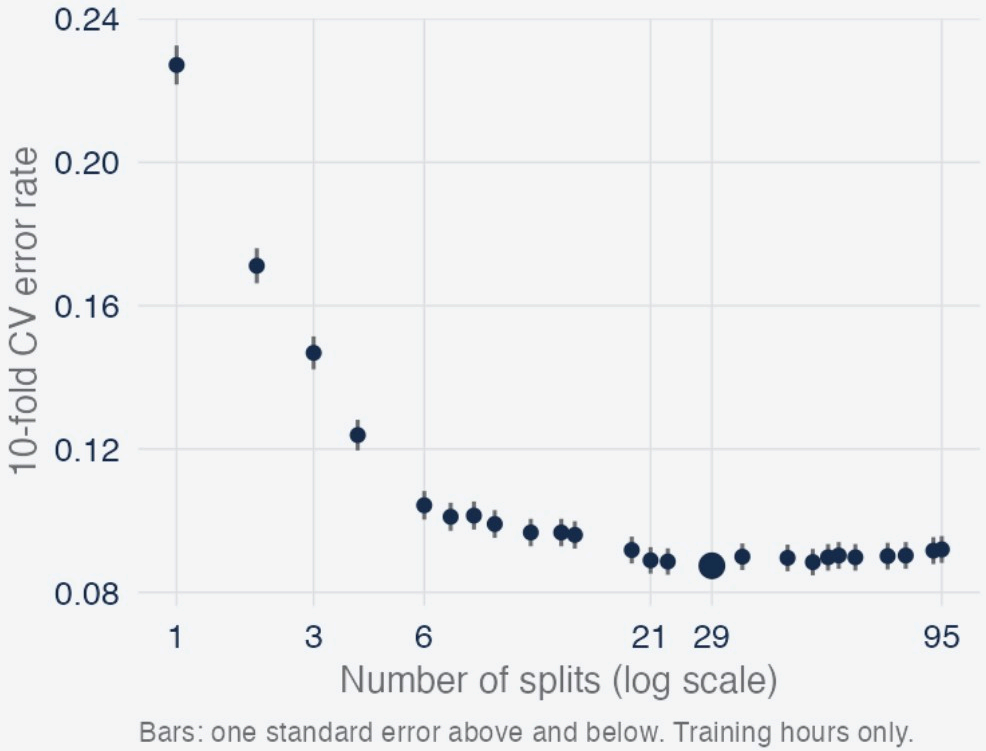
$$\text{error}(T) + \alpha\,|T|$$

Training error, plus a charge α per leaf. |T| is the number of leaves.

- **CP:** α scaled by the root error
- **nsplit:** number of splits
- **rel error:** training error ÷ root error
- **xerror:** CV error ÷ root error
- **xstd:** standard error of that CV score

- **We cannot use the red line to choose size.** Grow the full tree first (95 splits), then cut it back.
- **α is the price of one leaf.** A higher α keeps fewer leaves; α = 0 is the full tree.
- **Each α gives one subtree.** The table lists 26 trees, from 95 splits down to none. CP is rpart's scaled α, not α itself.

How does cross-validation choose the penalty?



Bars: one standard error above and below. Training hours only.

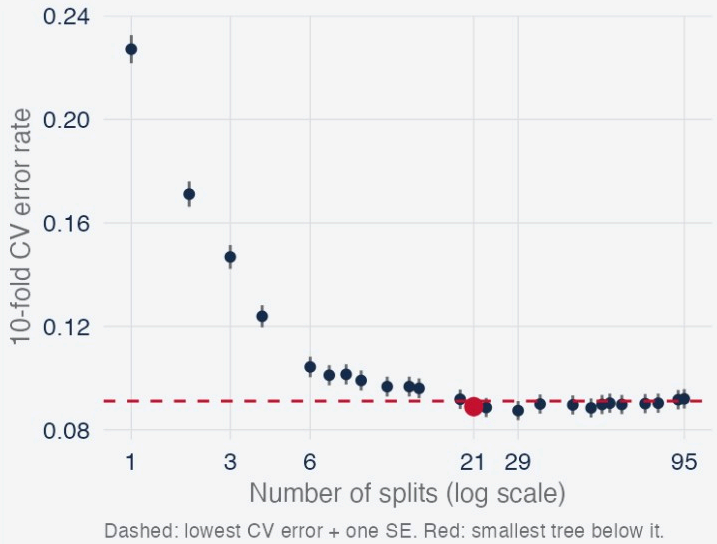
- **The dots are 10-fold CV error** on the training hours: each subtree scored on hours it did not see.
- **Lowest point: 29 splits,** CV error 0.0874.
- **At 21 splits:** 0.0889, which is 0.0015 higher.
- **The book's rule takes the minimum:** 29 splits.

29 splits or 21: which do we keep?

Why take the smallest tree within one standard error?

```r
cp   <- full$cptable                                  # one subtree per price
low  <- which.min(cp[, "xerror"])                     # subtree with the lowest CV error
thr  <- cp[low, "xerror"] + cp[low, "xstd"]           # lowest CV error plus one standard error
pick <- min(which(cp[, "xerror"] <= thr))             # smallest tree under that line
pruned <- prune(full, cp = cp[pick, "CP"] * (1 + 1e-9))  # prune the full tree to that subtree
cp[c(low, pick), c("CP", "nsplit", "xerror", "xstd")]    # the lowest subtree and the chosen subtree
```

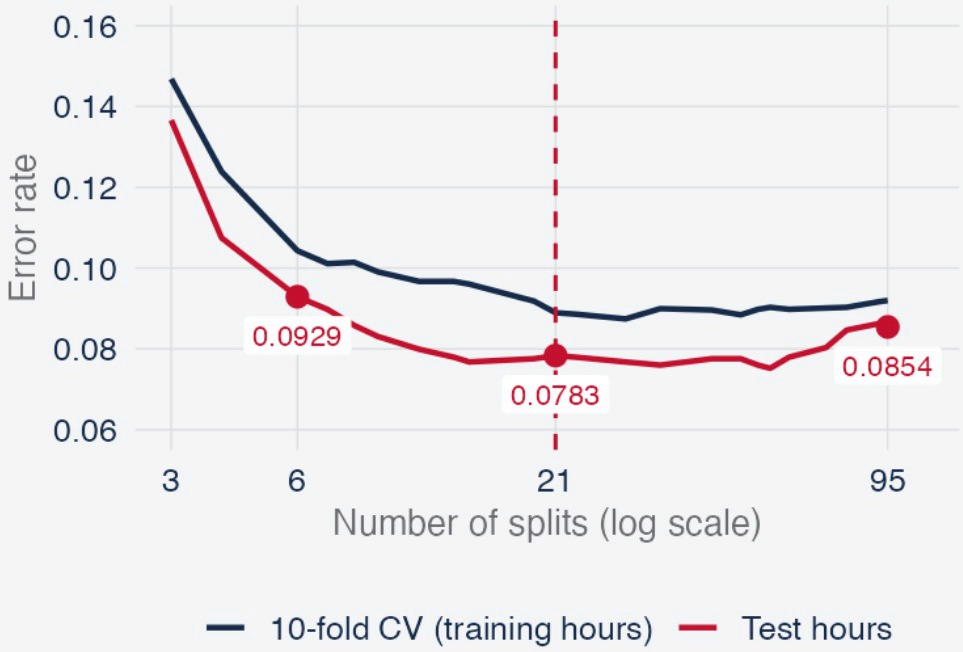
```
             CP nsplit    xerror        xstd
16 0.0009369876     29 0.1820098 0.007639483
14 0.0012297962     21 0.1851722 0.007699148
```



Dashed: lowest CV error + one SE. Red: smallest tree below it.

- **One-SE rule** (SE = standard error; Breiman, Friedman, Olshen and Stone): among trees within one SE of the lowest CV error, take the smallest.
- **Dashed line:** that threshold as an error rate, 0.0911.
- **First point under it:** 21 splits, 22 leaves, CP 0.00123.
- **Why 21, not 29:** easier to explain, and it moves less from sample to sample.

What did pruning buy us on the test hours?



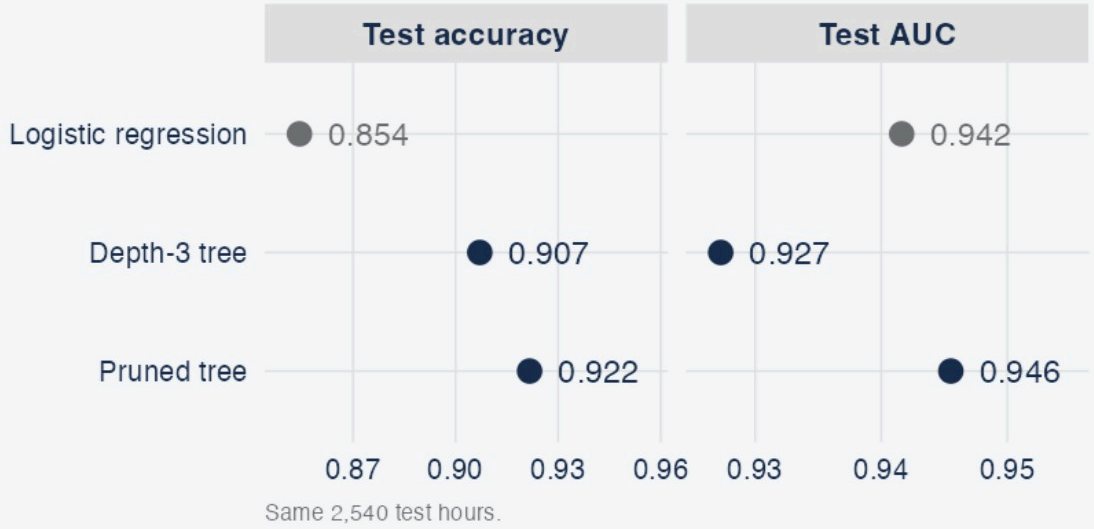
Red labels: test error. Dashed: the one-SE tree.

- **Marked test errors:** 0.0929 at 6 splits, 0.0783 at 21 (one-SE), 0.0854 at 95.
- **Size:** 21 splits instead of 95.
- **AUC:** full tree 0.970, pruned 0.946.

Does logistic regression beat the tree on the same hours?

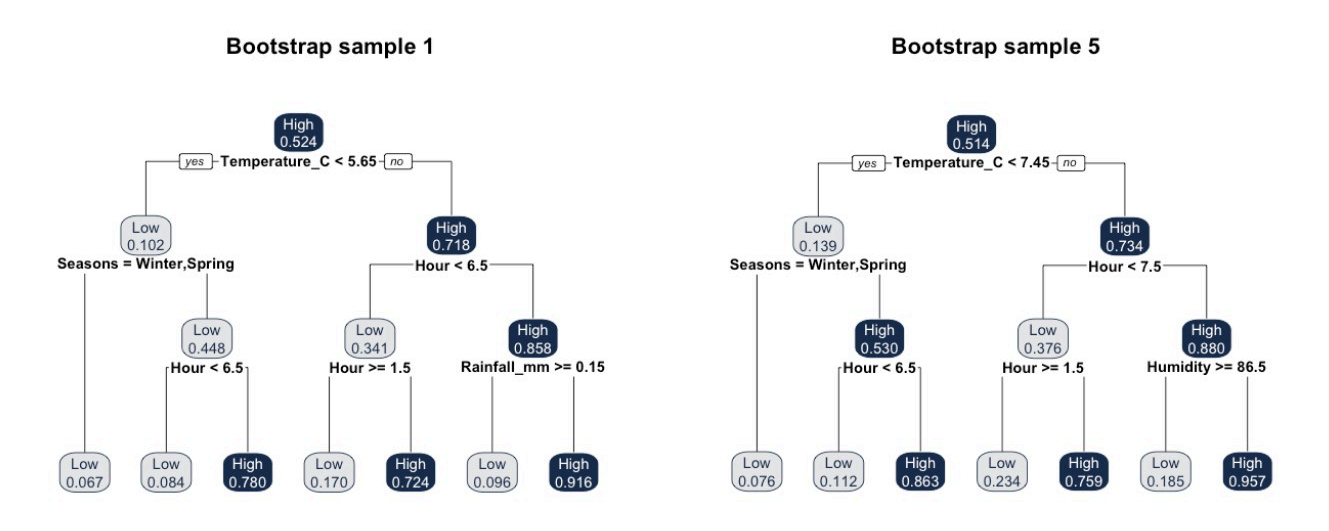
```
logit <- glm(High ~ ., data = train[, c("High", predictors)], family = binomial)  # logistic regression, same 10 predictor
p_logit <- predict(logit, test, type = "response")                                 # P(High) for each test hour
round(c(accuracy = acc_vec(test$High, p_logit), AUC = auc_vec(test$High, p_logit)), 3)  # same scores as the trees
```

```
accuracy      AUC
   0.854    0.942
```



- **Same 10 predictors, same test hours.** Logistic gives hour one straight-line effect; rentals have two peaks over the day (slide 8).
- **Left, accuracy:** logistic 0.854; depth-3 0.907; pruned 0.922. Both trees classify better than logistic.
- **Right, AUC:** depth-3 0.927 sits below logistic 0.942; pruned is highest at 0.946.

Would a slightly different sample give the same tree?



- **Bootstrap sample:** 5,925 hours drawn with replacement from the training hours.
- **5 samples, 5 root splits:** 5.65, 6.65, 6.65, 6.55 and 7.45°C.
- **2 different structures:** sample 1 uses rainfall where the others use humidity.
- **Test AUC from 0.916 to 0.935.**

What are the strengths and weaknesses of one tree?

Strengths

- Easy to explain: a few rules
- Close to business decision rules
- Good graphic displays
- Handles qualitative predictors, like season, without dummy variables

Weaknesses

- Accuracy can be lower; bagging, random forests and boosting raise it
- Not robust: a small change in the data can change the tree
- Not the best when the focus is on how a predictor relates to the outcome: the effect is a set of steps

What can we now say about one tree?

1. **What is a decision tree classifier?** Rules that split the hours into leaves; each leaf predicts its share of High hours.
2. **How are decision trees computed from the training data?** Greedy splits: the lowest RSS for a count, the lowest Gini for a class
3. **What are the big advantages and disadvantages of decision trees?** Readable rules; unstable and less accurate alone
4. **How can we minimize overfitting in tree models?** Grow a large tree, then prune it with 10-fold CV and the one-SE rule: 21 splits.

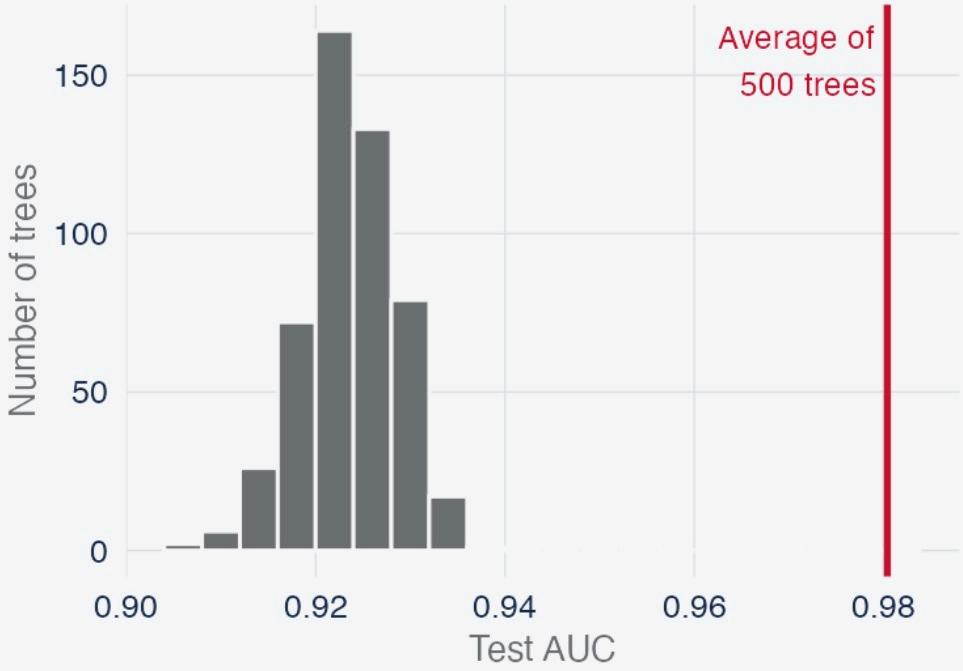
> If one tree is unstable, what if we grew many trees and averaged them?

Where do many trees sit on the course roadmap?

	Description	Tools	Use
PREDICT & EVALUATE			
Linear regression	Predict a number	OLS	
Metrics	Size of the errors	RMSE, R^2	
Train/test + leakage	Predict without fooling ourselves	Holdout	
Regularization	Shrink complexity	Ridge, lasso	
Cross-validation	Choose λ without touching test	k-fold	
CLASSIFY & DECIDE			
Logistic	A probability, not a class	Log-odds	
Thresholds	A score is not an action	ROC, threshold	
FLEXIBLE & UNSUPERVISED			
Trees	When linearity fails	Splits	Split the hours into simple regions.
Forests / boosting	**One ensemble**	**Random forest, boosting**	**Many trees, combined into one prediction.**
PCA	Compress X, no Y	PCs	
Clustering	Segment without Y	K-means	

> How can many trees predict better than one?

How does bagging beat one tree?



Bars: the 500 single trees of the bagged model. 2,540 test hours.

- **From Thursday's bootstrap trees:** five samples gave test AUC from 0.916 to 0.935. Now grow 500 deep trees, unpruned, and average their P(High).
- **Grey bars:** single trees, test AUC from 0.909 to 0.939 (median 0.923). **Red line:** their average, AUC 0.981.
- **An average varies less than one tree.** The trees are not independent, so the gain is smaller than for a mean of independent scores; the less alike the trees, the more averaging helps.

How can many trees predict better than one?

Bagging

Average deep trees

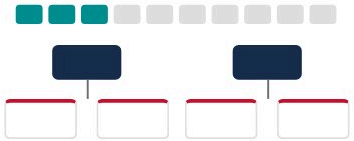


mean P(High)

Deep trees on bootstrap samples; average. (Previous slide.)

Random forest

Average deep trees that differ



mean P(High)

Like bagging, but each split draws a few predictors by lot.

Why would hiding predictors help?

Boosting

Add small trees, slowly



Small trees in sequence; each fits what is left, a small step at a time.

Which of the three should win on Seoul's test hours?

How does bagging build its 500 trees?

1 · Bagging | 2 · Random forest | 3 · Boosting

`data = train`	→	`num.trees = 500`	→	`mtry = 10`	→	`probability = TRUE`
5,925 training hours		500 bootstrap samples; ~37% of hours left out of each		One deep unpruned tree per sample; every split may try all 10 predictors		Average the 500 P(High); High if ≥ 0.5

```
bag <- ranger(High ~ ., data = train[, c("High", predictors)],    # many trees on the 10 predictors
              num.trees = 500,                                     # 500 bootstrap samples, one deep tree each
              mtry = 10,                                           # every split may try all 10 predictors: bagging
              probability = TRUE,                                  # each tree gives P(High); the forest averages them
              importance = "permutation", keep.inbag = TRUE,       # importance (later) and which hours each tree saw
              seed = 467, num.threads = 1)                         # same trees every run
p_bag <- predict(bag, test, num.threads = 1)$predictions[, "High"]  # average P(High) for each test hour
round(c(error = 1 - acc_vec(test$High, p_bag), accuracy = acc_vec(test$High, p_bag), AUC = auc_vec(test$High, p_bag)), 3)
```

```
  error accuracy      AUC
  0.070    0.930    0.981
```

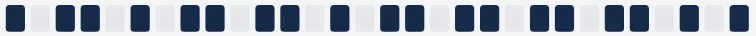
- **Bagging is a random forest that may try all 10 predictors at every split.**
- **Test:** error 0.070, accuracy 0.930, AUC 0.981 — the red line on slide 29.

About a third of the training hours are left out of each tree. Can we use them to estimate the test error?

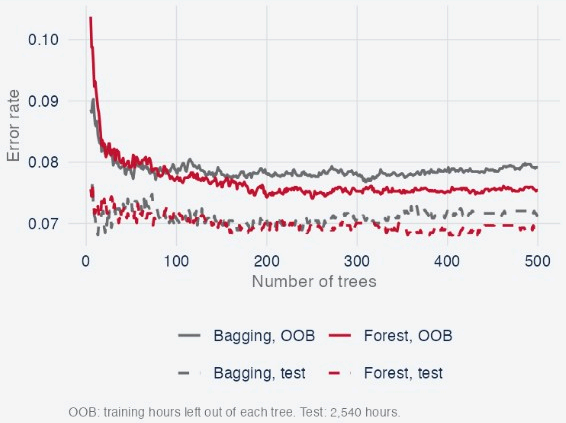
How can we measure test error without the test hours?

1 · Bagging | 2 · Random forest | 3 · Boosting

One training hour across the 500 trees



Grey: trees that left this hour out (~36.8%). Average only those = OOB.



OOB: training hours left out of each tree. Test: 2,540 hours.

$$\Pr(\text{hour missed}) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n\to\infty} \frac{1}{e} \approx 0.368$$

Each tree draws n hours with replacement from n. Here that share is 36.8%.

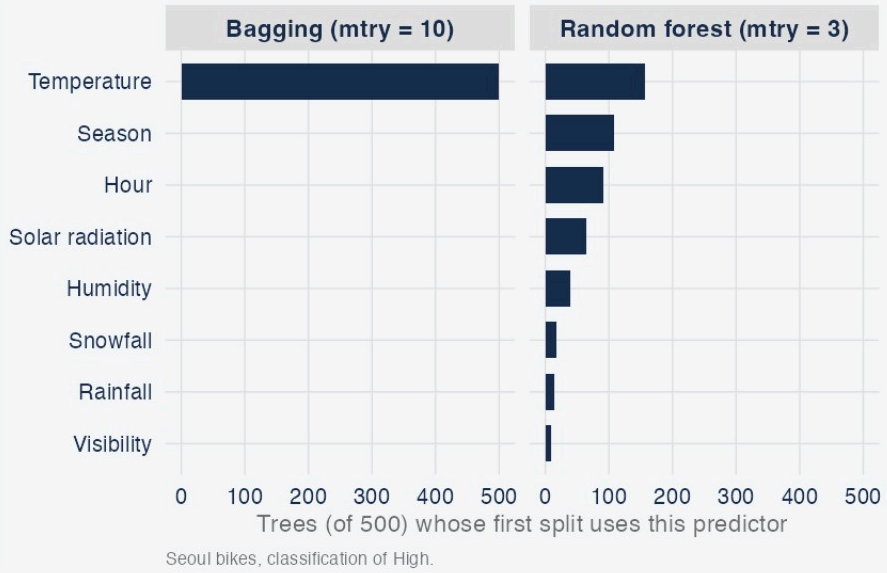
- **OOB (out-of-bag):** score each training hour only with trees that left it out; that estimates test error.
- **Grey is bagging:** OOB error 0.080 against test 0.070. Red is the forest; next block.
- **More trees do not overfit:** OOB error 0.099, 0.092, 0.080, 0.080 after 1, 10, 100, 500 trees.

Why do bagged trees look alike, and what does a forest change?

1 · Bagging | **2 · Random forest** | 3 · Boosting

Bagging	Temp	Hour	Season	Humidity	Wind	Vis	Solar	Rain	Snow	Holiday
Forest	Temp	Hour	Season	Humidity	Wind	Vis	Solar	Rain	Snow	Holiday

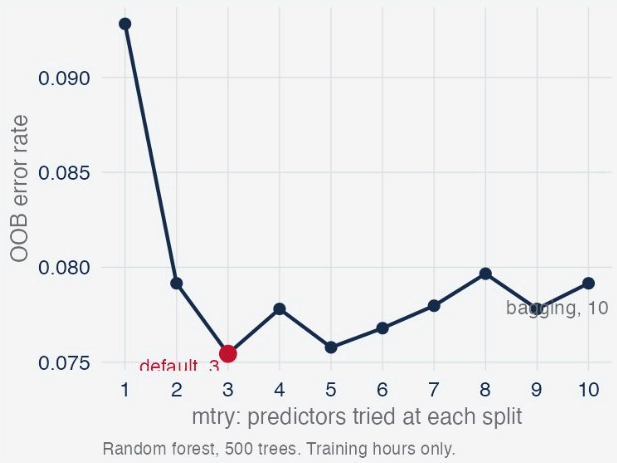
Temp not in this draw → another predictor goes first



- **Bagging:** temperature is the first split in 99.8% of 500 trees.
- **Averaging near-copies helps less** than averaging trees that disagree (slide 29).
- **Forest:** each split tries 3 of 10 predictors by lot; temperature first in 32.6%; 8 different starters against 2.

How many predictors should each split try?

1 · Bagging | **2 · Random forest** | 3 · Boosting



One argument changes: mtry falls to its default ⌊√10⌋ = 3

```
rf <- ranger(High ~ ., data = train[, c("High", predictors)],
             num.trees = 500, probability = TRUE,
             importance = "permutation", keep.inbag = TRUE,
             seed = 467, num.threads = 1)
p_rf <- predict(rf, test, num.threads = 1)$predictions[, "High"]
rf_oob <- mean((rf$predictions[, "High"] >= 0.5) !=
                 (train$High == "High"))
cat("mtry", rf$mtry, "\n")
round(c(OOB_error = rf_oob,
        test_error = 1 - acc_vec(test$High, p_rf),
        AUC = auc_vec(test$High, p_rf)), 3)
```

```
mtry 3
 OOB_error test_error        AUC
     0.077      0.068      0.984
```

- **Choose mtry on OOB error, not test error.** On the curve, 1 is too random, 10 is bagging, and 3 sits near the bottom.
- **Forest, mtry 3:** OOB error 0.077; test error 0.068, accuracy 0.932, AUC 0.984.

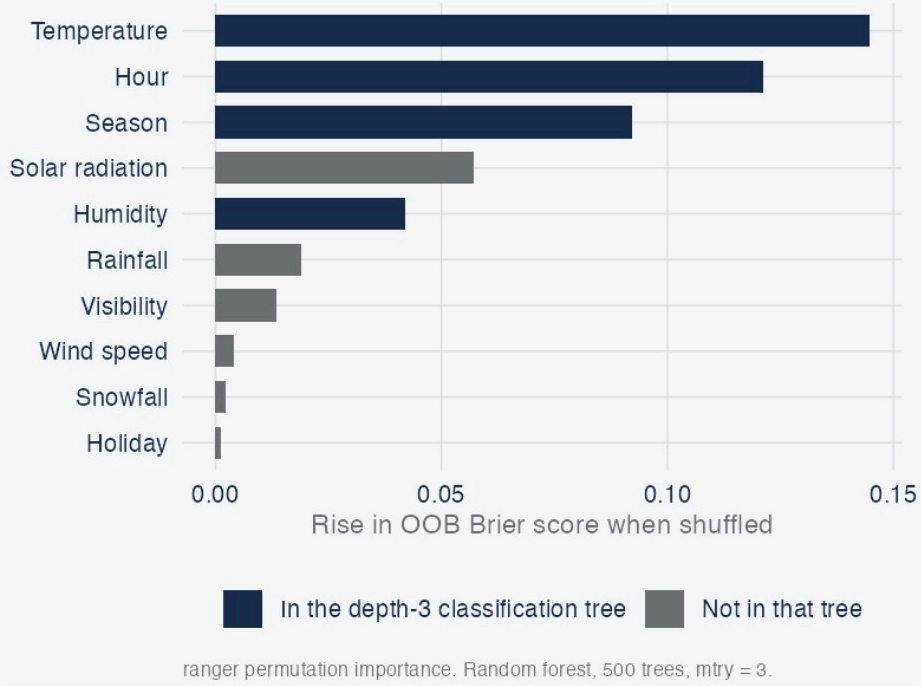
The curve is lowest at 4. Why keep 3?

Which variables matter to the forest?

1 · Bagging | **2 · Random forest** | 3 · Boosting

1. Take each tree’s out-of-bag hours.
2. Shuffle one column, say temperature.
3. Measure how much worse the OOB predictions get.

A big rise means the forest leaned on that predictor.



- **Temperature 0.142, hour 0.123, then season.** Rainfall ranks 6th of 10 (0.019): used by the forest, not by the depth-3 tree.

How does boosting build trees one after another?

1 · Bagging | 2 · Random forest | **3 · Boosting**

Bagging



Deep trees side by side; average P(High).

Boosting



Small trees in sequence; each fits what is left.

1 · Start
Predict the training mean, 732.5, for every hour. Leftover = rentals − 732.5.

→

2 · Fit
A small tree to the leftovers, not to the rentals.

→

3 · Add
A small step of that tree to the prediction so far.

→

4 · Update
Recompute leftovers, then go back to step 2.

$$r_i = y_i - \hat{f}(x_i) \qquad \hat{f}^{(b)} \text{ fits } (x_i, r_i) \qquad \hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^{(b)}(x)$$

(r_i): leftover. (y_i): rentals. (f): prediction so far. (f^{(b)}): tree on leftovers ((b) = tree number). (): step size. Count model; ISLR starts at 0, we start at the mean.

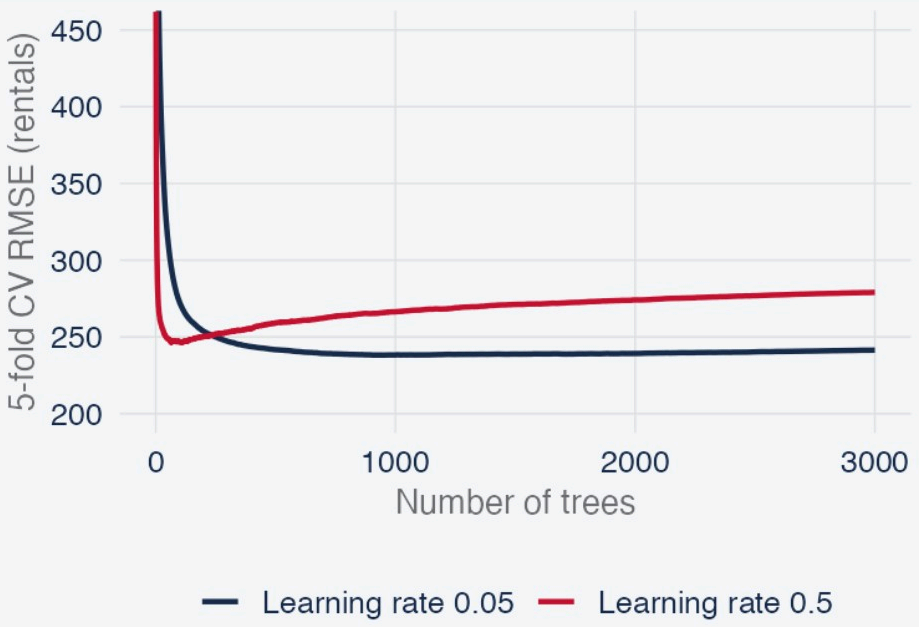
Mean only	1 tree	10	100	1,000
647.2 →	626.0 →	486.8 →	262.5 →	197.3

Training RMSE (rentals). Depth 3; λ = 0.05.

Bagging averages deep trees side by side. Boosting adds small trees in sequence, each fit to what earlier trees still miss.

What do the step size, the number of trees, and the depth do?

1 · Bagging | 2 · Random forest | **3 · Boosting**



Count model, depth 3. Training hours only.

λ · step size `eta`
Each tree adds only λ of its fit. Smaller steps need more trees.

B · number of trees `nrounds`
More trees can hurt. CV chooses B together with λ.

Depth `max_depth`
At most 3 splits along any path, so a tree can combine up to 3 predictors.

- **Rate 0.5 (red):** lowest CV RMSE 246.4 at 106 trees, then 279.0 by 3,000.
- **Rate 0.05 (navy):** slower and lower: 238.1 at 949 trees; 241.4 at 3,000.
- **Unlike bagging, more trees can hurt.** Pick B by CV with λ.

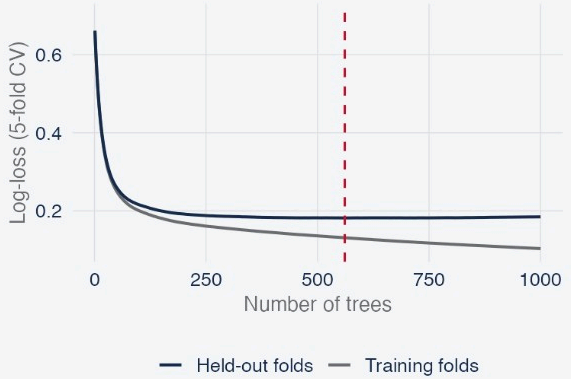
How do we estimate boosting in R, and how many trees?

1 · Bagging | 2 · Random forest | **3 · Boosting**

eta = λ | **nrounds** = B | **max_depth** = depth

```
x_tr <- train[, predictors]; x_te <- test[, predictors]            # xgboost 3 take
set.seed(467)                                                      # the 5 CV folds
cv <- xgb.cv(params = list(objective = "binary:logistic",          # predict P(High
                           eval_metric = "logloss",                # ... and score
                           learning_rate = 0.05, max_depth = 3,    # small steps, s
                           nthread = 1),                           # one thread: sa
             data = xgb.DMatrix(x_tr, label = as.numeric(train$High == "High")),  #
             nrounds = 1000, nfold = 5, verbose = 0)               # up to 1,000 tr
best <- which.min(cv$evaluation_log$test_logloss_mean)             # trees with the
set.seed(467)                                                      # same seed for
boost <- xgboost(x = x_tr, y = train$High, nrounds = best,         # estimate agair
                 learning_rate = 0.05, max_depth = 3, nthreads = 1)  # same setting
p_boost <- predict(boost, x_te)                                    # P(High) for ea
cat("trees", best, "\n")
round(c(accuracy = acc_vec(test$High, p_boost),
        AUC = auc_vec(test$High, p_boost)), 3)
```

```
trees 561
accuracy       AUC
   0.931     0.986
```



Dashed: number of trees with the lowest held-out log-loss.

- **Same loop, High or Low.** Leftover = 1 or 0 − P(High); score by log-loss.
- **5-fold CV:** lowest 0.1820 at 561 trees; 0.1849 at 1,000.
- **Refit at 561:** test accuracy 0.931, AUC 0.986.

What did each method change, and what did it buy?

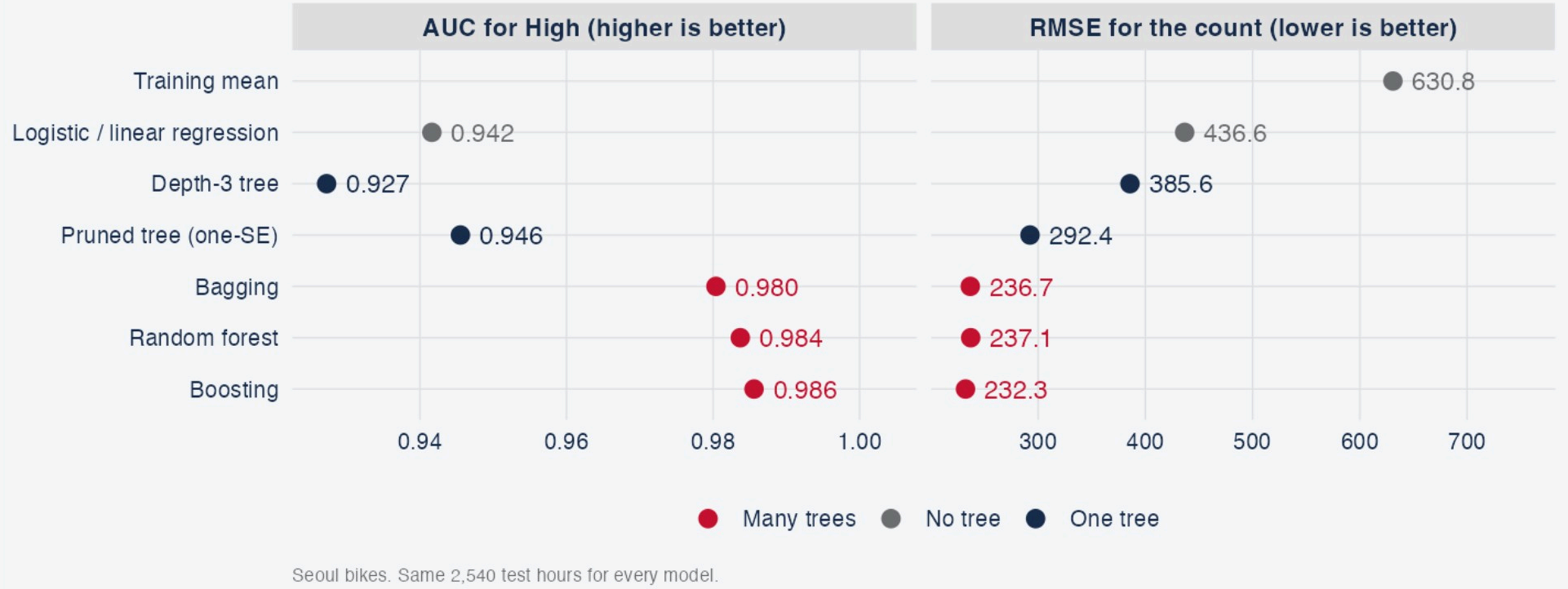
1 · Bagging | 2 · Random forest | 3 · Boosting

	Bagging	Random forest	Boosting
Hours each tree sees	Bootstrap sample	Bootstrap sample	All training hours
Tree size	Deep, unpruned	Deep, unpruned	Small, depth 3
Predictors a split may try	All 10	3 by lot	All 10
How trees combine	Average P(High)	Average P(High)	Add $\lambda = 0.05$ of each, in sequence
Setting on training hours	B = 500; more never hurts	mtry = 3 on OOB	B = 561 by 5-fold CV; more can hurt

Which of the three should win on Seoul's test hours?

Which model predicts best on the same test hours?

1 · Bagging | 2 · Random forest | 3 · Boosting



Seoul bikes. Same 2,540 test hours for every model.

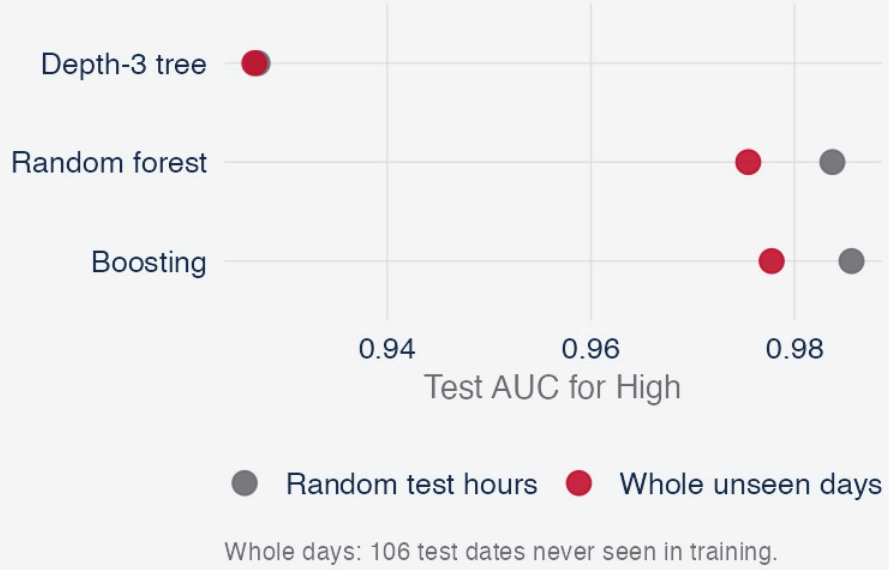
- **The ensembles beat one tree and are close to each other:** AUC 0.981, 0.984 and 0.986 against 0.946 for the pruned tree; RMSE 236.7, 235.9 and 232.3 against 292.4.
- **Every setting was chosen on the training hours. The test hours only score.**

Do the scores hold for days the model has not seen?

1 · Bagging | 2 · Random forest | 3 · Boosting

```
set.seed(467)                                                    # same seed
dsplit <- group_initial_split(bikes, group = date, prop = 0.70)  # whole days go to training or to test
dtr <- training(dsplit); dte <- testing(dsplit)                  # no date in both parts
c(train = nrow(dtr), test = nrow(dte), test_days = length(unique(dte$date)), shared = length(intersect(dtr$date, dte$date)))
```

```
   train     test test_days    shared
    5921     2544       106         0
```



Whole days: 106 test dates never seen in training.

- **Neighbouring hours are alike:** rentals in consecutive open hours correlate 0.902.
- **Every test date is also a training date,** and 89.6% of test hours have a neighbouring hour from the same day in training.
- **Whole-day split:** no date in both parts. On whole unseen days the depth-3 tree's AUC is 0.927; the forest and boosting lose a little and still lead.

What will we answer in the lab?

PROPOSED LAB QUESTION

> On whole days the model has never seen, which hours will be busy (at least 500 rentals), and how many rentals should we expect? Does a forest or boosting beat one pruned tree when the settings are chosen without those days?

What should we check before trusting a tree?

1. **Is a tree appropriate?** Yes: rentals have two peaks over the day, which a straight line misses. Slide 8
2. **Is this tree overfit?** The full tree is: 95 splits, and a higher test error than the pruned tree. Slide 19
3. **Was it pruned by cross-validation, and were ensembles tried?** Pruned with 10-fold CV and the one-SE rule; bagging, a random forest and boosting were tried. Slides 22 and 40
4. **Is accuracy reported on data used neither to grow the tree nor to choose it?** Yes: the test hours only score, and whole unseen days are checked too. Slides 40 and 41

How many bikes will Seoul riders rent in the next hour?

1. **One tree says why.** Warm hours from 7am are High unless humidity is 87% or more; cold winter and spring hours are Low (the depth-3 tree, slide 18).
2. **Many trees predict best** on the horse-race and whole-day slides; boosting leads narrowly.
3. **The trade-off:** one tree can be read; an ensemble predicts better.

How does training RMSE fall as boosting adds trees?

BACKUP: COUNT MODEL PATH FROM SLIDE 36



Boosting the count: depth 3, learning rate 0.05.

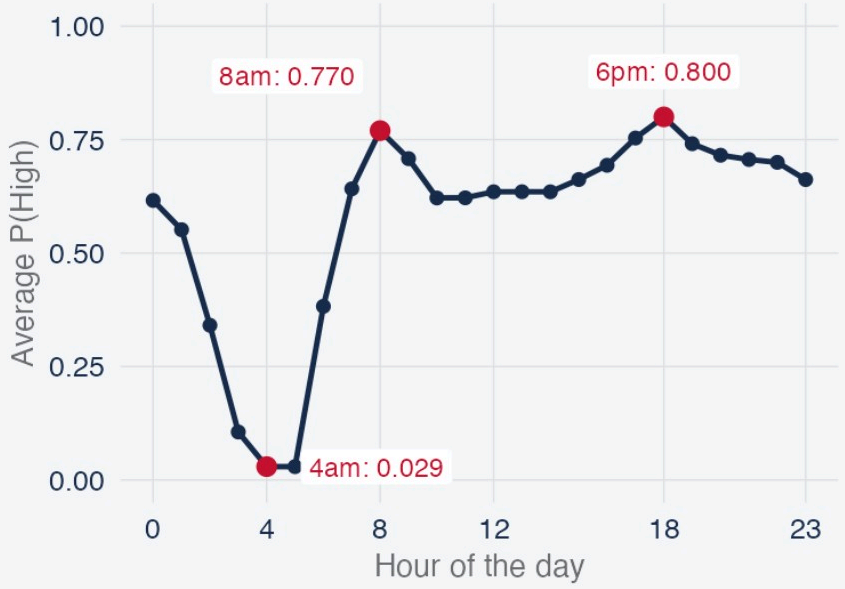
- **Same path as the RMSE strip on the mechanism slide**, now as a curve. Training: 626.0 → 197.3. Test: 610.7 → 232.1.
- **Depth 3; λ = 0.05.** For High or Low, CV later stops short of 1,000 trees.

How does the hour move the boosted prediction?

BACKUP: THE TEXTBOOK'S LAB, SECTION 8.3.4

```
pd_hour <- partial(boost, pred.var = "Hour", train = x_tr,           # set every training hour's Hour to 0, 1, ..., 23
                   pred.fun = function(object, newdata) mean(predict(object, newdata)),  # average P(High) at each value
                   grid.resolution = 24)                              # one point per hour of the day
round(pd_hour$yhat[pd_hour$Hour %in% c(4, 8, 18)], 3)               # 4am, 8am and 6pm
```

```
[1] 0.029 0.770 0.800
```



Boosting. Other predictors at their observed values.

- **Partial dependence:** set every training hour's Hour to one value, keep the other predictors as observed, and average the predicted P(High).
- **4am: 0.029. 8am: 0.770. 6pm: 0.800.**
- **The two peaks of slide 8 survive inside 561 trees.**